Customer Support Ticket Analyzer

Python Fundamentals - Module End Assignment

Problem Statement: Support teams get a lot of tickets every day and it's hard to manually go through all of them to see what's going wrong. This notebook loads ticket data, cleans up the issue descriptions, and pulls out some basic insights (priority split, keyword counts, longest complaint, unique words used).

Load the tickets

Step 1 - Load the tickets

```
[1]
```

```
ticket_data = {
    'Ticket_No': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Customer_Name': ['Ravi', 'Meera', 'Sam', 'Anu', 'Rakesh', 'Divya', 'Arjun', 'Kiran', 'Leela', 'Nisha'],
    'Issue_Description': [
        ' Internet not working!!! ',
        'slow response, very poor service ',
        'GREAT support! issue resolved.',
        ' okay... need help ',
        'not BAD but slow',
        'Excellent guidance, Very Helpful!',
        'good support and good behaviour!',
        'Poor handling of technical issue',
        'Satisfied. Could be better.',
        'Good service... quick response.'
    ],
    'Priority': ['High', 'Low', 'High', 'Medium', 'Low', 'High', 'Medium', 'High', 'Low', 'Medium']
}

# quick check
print(ticket_data)
```

```
{'Ticket_No': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 'Customer_Name': ['Ravi', 'Meera', 'Sam', 'Anu', 'Rakesh', 'Divya', 'Arjun', 'Kiran', 'Leela', 'Nisha'], 'Issue_Description': [' Internet not work
```

```
[2]
```

```
# function to print the dict of lists as a table, easier to read than the raw dict
def print_ticket_data(data, heading="TICKET DATA"):
    print("\n" + "=" * 70)
    print(heading.center(70))
    print("=" * 70)
    n = len(data['Ticket_No'])
    print(f"{'No':<4}{'Customer':<12}{'Priority':<10}{'Issue Description'}")
    print("-" * 70)
    for i in range(n):
        print(f"{data['Ticket_No'][i]:<4}{data['Customer_Name'][i]:<12}{data['Priority'][i]:<10}{data['Issue_Description'][i]}")
    print("=" * 70 + "\n")

print_ticket_data(ticket_data, "STEP 1: INITIAL TICKET DATA")
```

```
======================================================================
                    STEP 1: INITIAL TICKET DATA
======================================================================
No  Customer    Priority  Issue Description
----------------------------------------------------------------------
1   Ravi        High       Internet not working!!!
2   Meera       Low       slow response, very poor service
3   Sam         High      GREAT support! issue resolved.
4   Anu         Medium     okay... need help
5   Rakesh      Low       not BAD but slow
6   Divya       High      Excellent guidance, Very Helpful!
7   Arjun       Medium    good support and good behaviour!
8   Kiran       High      Poor handling of technical issue
9   Leela       Low       Satisfied. Could be better.
10  Nisha       Medium    Good service... quick response.
======================================================================
```

Add new tickets

Step 2 – Add new tickets

Asks how many tickets to add, then takes name/issue/priority for each one. Ticket numbers just keep counting up from whatever the last one was (so 11, 12, 13...). Priority input keeps asking again if it's not High/Medium/Low.

```
[5]
```

```
def add_new_tickets(data):
    try:
        num_new = int(input("How many new tickets do you want to add? "))
    except ValueError:
        print("that's not a number, skipping")
        return data

    next_no = max(data['Ticket_No']) + 1

    for i in range(num_new):
        print(f"\n--- New Ticket #{i + 1} ---")
        name = input("Customer Name: ").strip()
        issue = input("Issue Description: ").strip()

        while True:
            priority = input("Priority (High / Medium / Low): ").strip().capitalize()
            if priority in ('High', 'Medium', 'Low'):
                break
            print("that's not valid, try again")

        data['Ticket_No'].append(next_no)
        data['Customer_Name'].append(name)
        data['Issue_Description'].append(issue)
        data['Priority'].append(priority)
        next_no += 1

    return data

# uncomment this if you actually want to type in new tickets when running the notebook
# ticket_data = add_new_tickets(ticket_data)
```

Clean up the issue descriptions

Step 3 - Clean up the issue descriptions

Removing punctuation, extra spaces, making everything lowercase, and swapping out a few shorthand words like "ok" -> "okay" and "bad" -> "poor".

[6] ✓ 0s

```
punct_chars = ".,!?-"

slang = {
    'ok': 'okay',
    'bad': 'poor',
    'v': 'very',
    'pls': 'please',
    'thx': 'thanks',
}

def clean_text(text):
    for ch in punct_chars:
        text = text.replace(ch, '')
    text = text.lower()
    text = ' '.join(text.split())  # gets rid of double spaces / leading-trailing spaces
    text = ' '.join(slang.get(w, w) for w in text.split())
    return text

ticket_data['Issue_Description'] = [clean_text(d) for d in ticket_data['Issue_Description']]
print_ticket_data(ticket_data, "STEP 3: AFTER CLEANING")
```

```
======================================================================
                      STEP 3: AFTER CLEANING
======================================================================
No  Customer    Priority  Issue Description
----------------------------------------------------------------------
1   Ravi        High      internet not working
2   Meera       Low       slow response very poor service
3   Sam         High      great support issue resolved
4   Anu         Medium    okay need help
5   Rakesh      Low       not poor but slow
6   Divya       High      excellent guidance very helpful
7   Arjun       Medium    good support and good behaviour
8   Kiran       High      poor handling of technical issue
9   Leela       Low       satisfied could be better
10  Nisha       Medium    good service quick response
======================================================================
```

Keyword search

Step 4 - Keyword search

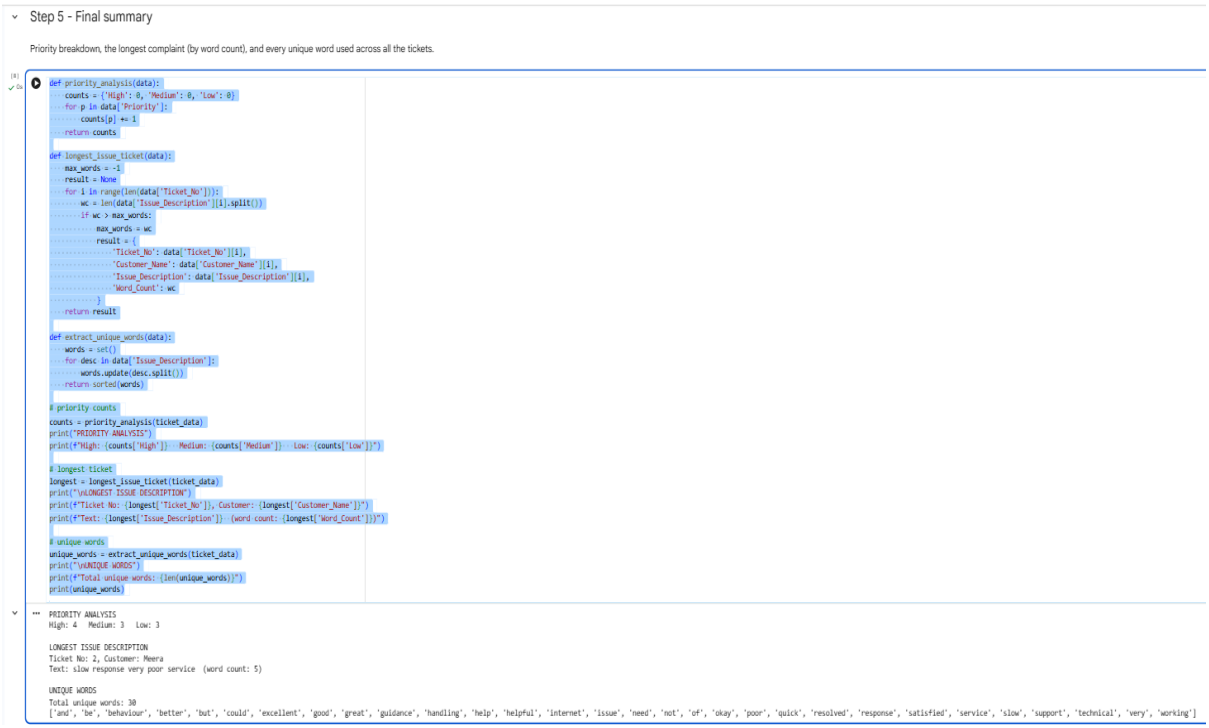
[7] ✓ 0s

```
def count_tickets_with_word(word, data=ticket_data):
    word = word.lower()
    count = 0
    for desc in data['Issue_Description']:
        if word in desc.split():
            count += 1
    return count

for kw in ['poor', 'good', 'slow', 'excellent']:
    print(f"Tickets containing '{kw}': {count_tickets_with_word(kw)}")
```

```
Tickets containing 'poor': 3
Tickets containing 'good': 2
Tickets containing 'slow': 2
Tickets containing 'excellent': 1
```

Final summary



Summary & Insights

Out of the 10 tickets, 4 are High priority, 3 Medium and 3 Low, so a good chunk of the issues coming in need urgent attention. Looking at the keyword counts, both positive words (good, excellent) and negative ones (poor, slow) show up, so customer sentiment is mixed rather than clearly good or bad. The longest complaint was Meera's ticket ("slow response very poor service"), which also happens to be one of the more negative ones - longer complaints might be worth flagging for review first. There are 30 unique words across all the cleaned descriptions, which gives a decent picture of the common themes (slow, poor, good, service, support) that keep coming up and could be used later for tagging tickets automatically